

Taskify Backend Documentation

1. Project Overview

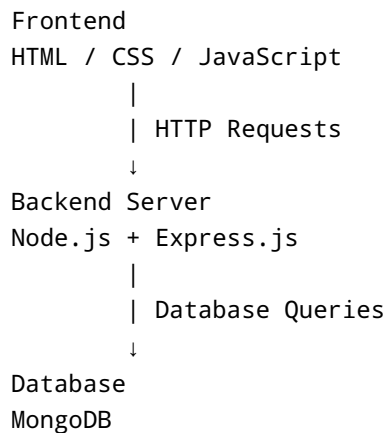
Taskify is a full-stack web application designed to help users manage their daily tasks. The system allows users to create, view, update, complete, and delete tasks in an organized way.

The backend of Taskify is responsible for handling user authentication, task management, data validation, and communication with the database. It provides secure API endpoints that the frontend can use to send and receive data.

The backend runs locally because the project is not expected to be hosted online.

2. Backend System Architecture

Taskify uses a simple client-server architecture.



Main Backend Responsibilities

The backend handles:

1. User registration
2. User login
3. Password hashing
4. Authentication using JWT
5. Creating tasks
6. Reading tasks
7. Updating tasks
8. Deleting tasks

- 9. Validating user input
 - 10. Storing and retrieving data from the database
-

3. Technology Stack

```
Runtime: Node.js
Framework: Express.js
Database: MongoDB
Authentication: JWT
Password Security: bcrypt
Environment Variables: dotenv
API Testing: Postman / Thunder Client
```

4. Backend Folder Structure

```
backend/
|
├─ server.js
├─ package.json
├─ .env
|
├─ config/
|   └─ db.js
|
├─ models/
|   ├── User.js
|   └─ Task.js
|
├─ routes/
|   ├── authRoutes.js
|   └─ taskRoutes.js
|
├─ controllers/
|   ├── authController.js
|   └─ taskController.js
|
├─ middleware/
|   └─ authMiddleware.js
```

5. Database Design

User Model

The User model stores account details for each user.

```
User
- id
- name
- email
- password
- createdAt
```

Task Model

The Task model stores task details created by users.

```
Task
- id
- userId
- title
- description
- dueDate
- priority
- status
- createdAt
- updatedAt
```

Each task belongs to one user through the `userId` field.

6. API Route Design

Authentication Routes

```
POST /api/auth/register
POST /api/auth/login
```

Task Routes

```
GET    /api/tasks
POST   /api/tasks
```

```
GET    /api/tasks/:id
PUT    /api/tasks/:id
DELETE /api/tasks/:id
```

7. Authentication System

Taskify uses JWT authentication.

When a user logs in successfully, the backend generates a token. This token is sent to the frontend and stored locally. For protected routes, the frontend sends the token back to the backend.

```
User logs in
      ↓
Backend checks email and password
      ↓
Backend creates JWT token
      ↓
Frontend stores token
      ↓
Frontend uses token to access task routes
```

Protected task routes require a valid token.

8. Validation Rules

Register Validation

- Name is required
- Email is required
- Password is required
- Email must not already exist

Login Validation

- Email is required
- Password is required
- User must exist
- Password must be correct

Task Validation

- Title is required
- Description is required
- Due date is required
- Priority is required
- Status is required

9. Task Status and Priority

Task Status

```
pending
completed
```

Task Priority

```
low
medium
high
```

10. Security Design

The backend includes basic security features:

- Passwords are hashed before saving
- JWT tokens protect private routes
- Users can only access their own tasks
- Input validation prevents empty fields
- Environment variables hide sensitive information

11. Example Environment Variables

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/taskify
JWT_SECRET=taskify_secret_key
```

12. Backend Workflow

Register Workflow

```
User submits name, email, and password
      ↓
Backend validates input
      ↓
Backend checks if email already exists
      ↓
Password is hashed
      ↓
User is saved in database
      ↓
Success response is returned
```

Login Workflow

```
User submits email and password
      ↓
Backend validates input
      ↓
Backend checks user exists
      ↓
Backend compares password
      ↓
JWT token is generated
      ↓
Token is returned to frontend
```

Create Task Workflow

```
User submits task details
      ↓
Backend verifies JWT token
      ↓
Backend validates task fields
      ↓
Task is saved with userId
      ↓
Task is returned to frontend
```

Get Tasks Workflow

```
Frontend requests tasks
    ↓
Backend verifies JWT token
    ↓
Backend finds tasks belonging to user
    ↓
Tasks are returned
```

Update Task Workflow

```
User edits task
    ↓
Backend verifies JWT token
    ↓
Backend checks task belongs to user
    ↓
Task is updated
    ↓
Updated task is returned
```

Delete Task Workflow

```
User clicks delete
    ↓
Backend verifies JWT token
    ↓
Backend checks task belongs to user
    ↓
Task is deleted
    ↓
Success message is returned
```

13. Expected Backend Responses

Success Response Example

```
{
  "success": true,
  "message": "Task created successfully",
}
```

```
"task": {  
  "title": "Complete assignment",  
  "status": "pending"  
}  
}
```

Error Response Example

```
{  
  "success": false,  
  "message": "Task title is required"  
}
```

14. Local Development Setup

Step 1: Create Backend Folder

```
mkdir backend  
cd backend
```

Step 2: Initialize Node Project

```
npm init -y
```

Step 3: Install Dependencies

```
npm install express mongoose bcryptjs jsonwebtoken dotenv cors
```

Step 4: Install Development Dependency

```
npm install nodemon --save-dev
```

Step 5: Run Backend Server

```
npm run dev
```

15. Conclusion

The Taskify backend provides the main logic and data management system for the application. It supports user authentication, task CRUD operations, validation, and secure database access.

This backend allows the frontend to communicate with the database through organized API endpoints, making Taskify a complete full-stack task management application.